

Creating custom packet using TCP flags

(Colasoft/scapy/Hping)

Packet Crafting is a technique to manually define the parameters of a packet and transmit it.

Allows you to:

- Test device behavior
- Audit firewall rules
- Ensure proper operation of security devices

Common Crafting Tools:

- Hping – <http://hping.org>
- Colasoft Packet Builder – <http://colasoft.com>

scapy

```
L$ sudo scapy
[sudo] password for kali:
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: GtkStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/ksm0'
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - Capture Start ...
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - Capture started
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - Version 2.5.0
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - https://github.com/secdev/scapy
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - Have fun!
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - Craft packets like I craft my beer.
** (Wireshark:6644): Gtk-WARNING **: 08:06:56.949000: [Capture:Wireshark] - -- Jean De Clerck

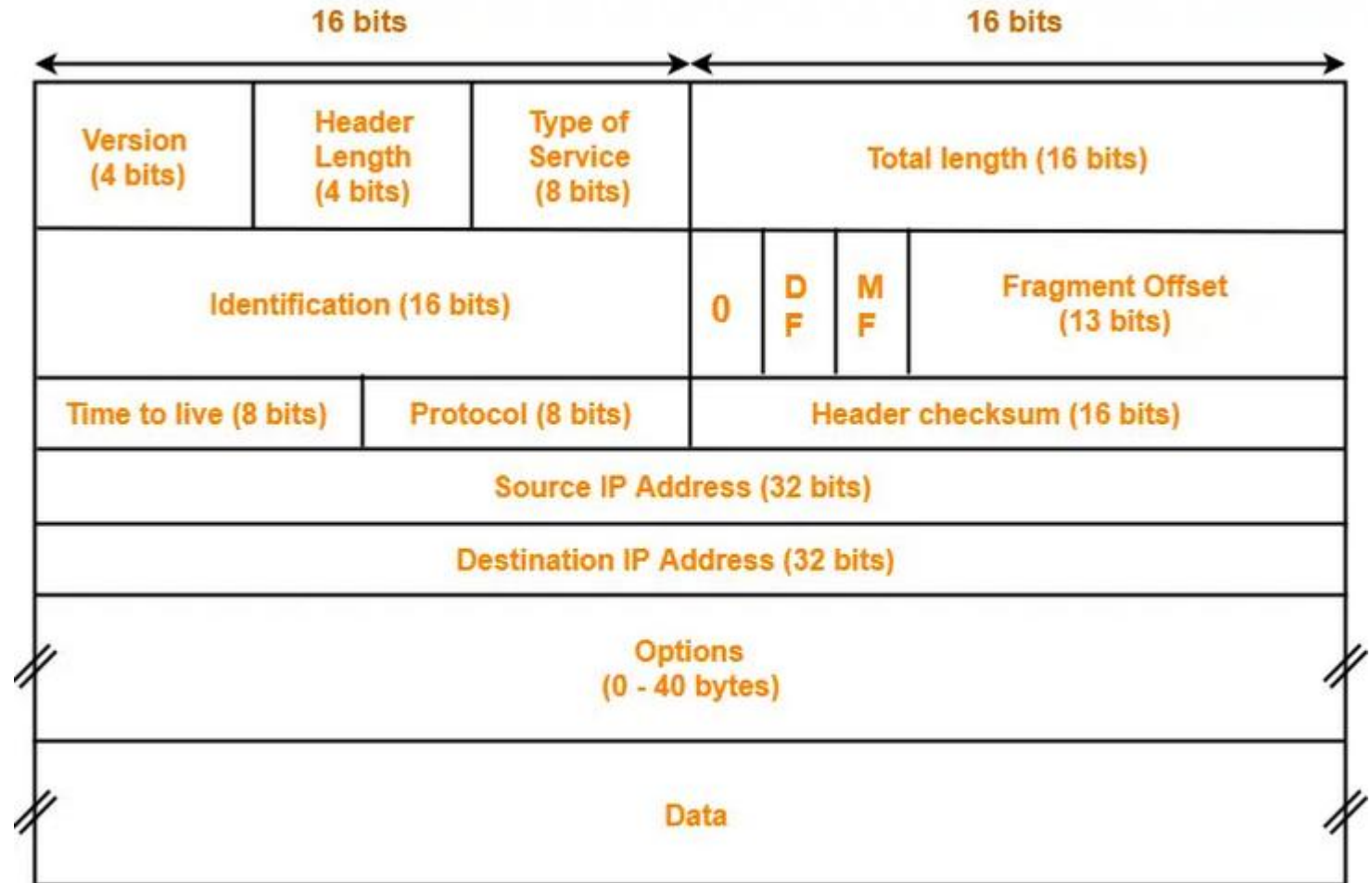
aSPY//YASa
apyyyyCY/////////YCa
sY/////////YSpcs scpCY//Pp
ayp ayyyyyyyySCP//Pp syY//C
AYAsAYYYYYYYY///Ps cY//S
pCCCCY//p cSSps y//Y
SPPPP///a pP///AC//Y
A//A cyP///C
p///Ac sC///a
P///YCpc A//A
sccccp///pSP///p p//Y
sY/////////y caa S//P
cayCyayP//Ya pY/Ya
sY/PsY/////////YCc aC//Yp
sc sccaCY//PCypaapyCP//YSs
spCPY/////////YPSps
ccaacs

using IPython 8.14.0
```

Change IP address by command

```
>>> IP().show()  
###[ IP ]###  
version= 4  
ihl= None  
tos= 0x0  
len= None  
id= 1  
flags=  
frag= 0  
ttl= 64  
proto= hopopt  
chksum= None  
src= 127.0.0.1  
dst= 127.0.0.1  
\options\  

```



IPv4 Header

Create IP packet

```
>>> a = IP(ttl=10)
>>> a
<IP ttl=10 ▸
>>> a.src
'127.0.0.1'
>>> a.dst = "192.168.30.129"
>>> a
<IP ttl=10 dst=192.168.30.129 ▸
>>> 
```

- Ttl by default =64 in linux
- Source ip address
- Destination ip address

IP on any protocol

- By default IP address on UDP or TCP
- We can specify the IP on any protocol work, here work on TCP

```
>>> IP()
<IP  >
>>> IP()/TCP()
<IP  frag=0 proto=tcp |<TCP  |>>
>>> 
```

specify the Ethernet and IP address

- Put depend on the layers
 - Layer2: Ether()
 - Layer3: IP()
 - Layer4:TCP()

```
>>> Ether()/IP()/TCP()
<Ether type=IPv4 |<IP frag=0 proto=tcp |<TCP |>>>
>>> |
```

Determine the header for protocol http

```
>>> IP()/TCP()/ "GET/HTTP/1.0\r\n\r\n"  
<IP frag=0 proto=tcp |<TCP |<Raw load='GET/HTTP/1.0\r\n\r\n' |>>>  
>>> |
```

homework

Design packet on each layer

List of layers

```
>>> ls()
```

```
ARP          : ARP
```

```
DHCP         : DHCP options
```

```
DNS          : DNS
```

```
Dot11        : 802.11
```

```
[...]
```

List of commands

List of commands

```
>>> lsc()
```

```
sr      : Send and receive packets at layer 3
```

```
sr1     : Send packets at layer 3 and return only the fi
```

```
srp     : Send and receive packets at layer 2
```

```
[...]
```

Records of IP headers

Example : Default Values for IP

```
>>> ls(IP)
```

version	: BitField	= (4)
ihl	: BitField	= (None)
tos	: XByteField	= (0)
len	: ShortField	= (None)
id	: ShortField	= (1)
flags	: FlagsField	= (0)
frag	: BitField	= (0)
ttl	: ByteField	= (64)
proto	: ByteEnumField	= (0)
checksum	: XShortField	= (None)
src	: Emph	= (None)
dst	: Emph	= ('127.0.0.1')
options	: IPOptionsField	= ('')

Packet Manipulation

Some stuff you can do on a packet

- `str(pkt)` to assemble the packet
- `hexdump(pkt)` to have an hexa dump
- `ls(pkt)` to have the list of fields values
- `pkt.summary()` for a one-line summary
- `pkt.show()` for a developed view of the packet
- `pkt.show2()` same as `show` but on the assembled packet (checksum is calculated, for instance)
- `pkt.sprintf()` fill a format string with fields values of the packet
- `pkt.decode_payload_as()` change the way the payload is decoded
- `pkt.haslayer()` test the presense of a layer
- `pkt.getlayer()` return a given layer

Show() and show2()

show() is suitable for quickly examining **packet contents**

show2() is useful for more **detailed analysis and troubleshooting**

```
>>> packet = IP(src="192.168.0.1", dst="192.168.0.2")
```

```
>>> packet.proto=1
>>> packet.show()
###[ IP ]###
  version = 486441 08:06:5
  ihl ser = None ing to /t
  tos wire = 0x0441 08:07:0
  len wire = None41 08:07:0
  id wire = 186441 08:07:0
  flags ocap =
  frag wire = 086441 08:02:3
  ttl wire = 646441 08:02:3
  proto w) = icmpce: 7095,
  checksum de = None
  src wire = 192.168.0.1 0:3
  dst      = 192.168.0.2
  \options \
```

```
>>> packet.show2()
###[ IP ]###
  version = 4
  ihl      = 5
  tos      = 0x0
  len      = 20
  id       = 1
  flags    =
  frag     = 0
  ttl      = 64
  proto    = icmp
  checksum = 0xf994
  src      = 192.168.0.1
  dst      = 192.168.0.2
  \options \
```

Haslayer() to ask about the packet

```
>>> pkt = IP()/TCP()  
>>> print("has Ether layer") if pkt.haslayer(Ether) else print("does not have Ether layer")  
does not have Ether layer  
>>> print("has Ether layer") if pkt.haslayer(TCP) else print("does not have Ether layer")  
has Ether layer  
>>> █
```

Get_layer functions

```
>>> pkt = Ether()/IP()/TCP()
>>> iplayer = pkt.getlayer(IP)
>>> iplayer.show() if iplayer else print("no ip layer found")
####[ IP ]####
  version m = 4 kali_notes
  ihl      = None
  tos      = 0x0
  len      = None
  id       = 1
  flags    =
  frag     = 0
  ttl      = 64
  proto    = tcp
  checksum = None
  src      = 127.0.0.1
  dst      = 127.0.0.1
  \options \
####[ TCP ]####
  sport = ftp_data
  dport = http
  seq    = 0
  ack    = 0
  dataofs = None
  reserved = 0
  flags   = S
  window  = 8192
  checksum = None
  urgptr  = 0
  options = []
```

Create IP address for Ethernet contain http header

```
>>> a=Ether()/IP(dst="www.slashdot.org")/TCP()/"GET /index.html HTTP/1.0 \n\n"
>>> hexdump(a)
0000  00 50 56 F4 90 B5 00 0C 29 97 24 72 08 00 45 00  .PV.....).$r..E.
0010  00 43 00 01 00 00 40 06 5F DC C0 A8 1E 81 CC 44  .C....@._.....D
0020  6F 6A 00 14 00 50 00 00 00 00 00 00 00 00 50 02  oj...P.....P.
0030  20 00 A2 D9 00 00 47 45 54 20 2F 69 6E 64 65 78  .....GET /index
0040  2E 68 74 6D 6C 20 48 54 54 50 2F 31 2E 30 20 0A  .html HTTP/1.0 .
0050  0A loading state information... Done
      E: Unable to locate package graph
>>> kali:~$
```

Spoofing(send packet with non-real ip address)

```
>>> send(IP(dst="192.168.30.132")/ICMP())
```

```
.  
Sent 1 packets.
```

```
>>> █
```

Send packet

```
>>> p = sr1(IP(dst="192.168.30.130", ttl=16)/TCP())
```

```
Begin emission:
```

```
Finished sending 1 packets.
```

```
.*
```

```
Received 2 packets, got 1 answers, remaining 0 packets
```

```
>>> c = arm
```

Send multi-packets

```
>>> p = sr1(IP(dst="192.168.30.130", ttl=16)/TCP())
Begin emission:
Finished sending 1 packets.
.*
Received 2 packets, got 1 answers, remaining 0 packets
>>> send(p)
.
Sent 1 packets.
>>> send([p]*10)
.....
Sent 10 packets.
>>> █
```

Make loop

```
>>> send(p, inter=0.1, loop=1)
```

```
.....^C
```

```
Sent 47 packets.
```

```
>>> ^C
```

Return only one packet receive

```
>>> sr1(IP(dst="192.168.30.130", ttl=4)/ICMP()/"hello")
Begin emission:
Finished sending 1 packets.
.*
Received 2 packets, got 1 answers, remaining 0 packets
<IP version=4 ihl=5 tos=0x0 len=33 id=7115 flags= frag=0 ttl=128 proto=icmp chksum=0x60bd sr
c=192.168.30.130 dst=192.168.30.129 |<ICMP type=echo-reply code=0 chksum=0xbc2d id=0x0 seq=0
x0 |<Raw load='hello' |<Padding load='\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
|>>>>
```

Sr1

```
>>> sr1(IP(dst="192.168.30.132")/TCP(dport=80, flags="S"))
Begin emission:
Finished sending 1 packets.
..*
Received 3 packets, got 1 answers, remaining 0 packets
<IP version=4 ihl=5 tos=0x0 len=44 id=714 flags=DF frag=0 ttl=128 proto=t
cp checksum=0x39ac src=192.168.30.132 dst=192.168.30.129 |<TCP sport=http d
port=ftp_data seq=2957441346 ack=1 dataofs=6 reserved=0 flags=SA window=81
92 checksum=0xfd2 urgptr=0 options=[('MSS', 1460)] |<Padding load='\x00\x00
' |>>>
>>> 
```

home work

BY OFFENSIVE SECURITY

Send to multi-ports

```
>>> sr(IP(dst="192.168.30.132")/TCP(sport= 6666, dport=(440, 443), flags="
... : S"))
Begin emission:
Finished sending 4 packets.
..****
Received 6 packets, got 4 answers, remaining 0 packets
(<Results: TCP:4 UDP:0 ICMP:0 Other:0>,
<Unanswered: TCP:0 UDP:0 ICMP:0 Other:0>)
>>> |
```

Send with data

```
>>> send(IP(dst="192.168.30.130", ttl=4)/ICMP()/ "hello")
.
Sent 1 packets.
>>> d= send(IP(dst="192.168.30.130", ttl=4)/ICMP()/ "hello")
.
Sent 1 packets.
```

Make sniff for ICMP, and make ping to capture packets

```
>>>  
>>> sniff(filter="icmp", count=2)  
<Sniffed: TCP:0 UDP:0 ICMP:2 Other:0  
>>> 
```

```
kali@kali:~$ ping 192.168.30.132  
PING 192.168.30.132 (192.168.30.132) 56(84) bytes of data.  
^C  
--- 192.168.30.132 ping statistics ---  
10 packets transmitted, 0 received, 100% packet  
loss, time 9225ms
```

```
kali@kali:~$
```

Sniffing for icmp and host

```
>>> sniff(filter="icmp and host 192.168.30.129", count=2)
<Sniffed: TCP:0 UDP:0 ICMP:2 Other:0>
>>> |
```

Use summary to collect information about
packets use number of lines =2

```
>>> sniff(count=2, prn=lambda x:x.summary())  
Ether / IP / UDP / DNS Qry "b'win10.ipv6.microsoft.com.'"  
Ether / IP / TCP 192.168.30.130:1604 > 10.10.10.10:1514 S  
<Sniffed: TCP:1 UDP:1 ICMP:0 Other:0>  
>>> ctrl-c
```

Collect information about the device send or receive from any devices

```
>>> sniff(iface="eth0", prn=lambda x: x.summary())
Ether / IP / UDP 192.168.30.132:netbios_ns > 192.168.30.2:netbios_ns / NBN
SQueryRequest / Raw
Ether / IP / UDP / DNS Qry "b'teredo.ipv6.microsoft.com.'"
Ether / IP / UDP 192.168.30.132:netbios_ns > 192.168.30.2:netbios_ns / NBN
SQueryRequest / Raw
Ether / IP / UDP / DNS Qry "b'teredo.ipv6.microsoft.com.'"
Ether / ARP who has 192.168.30.2 says 192.168.30.132 / Padding
Ether / ARP is at 00:50:56:f4:90:b5 says 192.168.30.2 / Padding
Ether / IP / UDP / DNS Qry "b'teredo.ipv6.microsoft.com.'"
Ether / IP / UDP 192.168.30.132:netbios_ns > 192.168.30.2:netbios_ns / NBN
SQueryRequest / Raw
Ether / IP / UDP 192.168.30.132:netbios_ns > 192.168.30.2:netbios_ns / NBN
SQueryRequest / Raw
```

Sniff with filter and lambda

```
>>> sniff(count=4, filter="icmp", prn=lambda x:x.summary())
Ether / IP / ICMP 192.168.30.129 > 192.168.30.130 echo-request 0 / Raw
Ether / IP / ICMP 192.168.30.130 > 192.168.30.129 echo-reply 0 / Raw
Ether / IP / ICMP 192.168.30.129 > 192.168.30.130 echo-request 0 / Raw
Ether / IP / ICMP 192.168.30.130 > 192.168.30.129 echo-reply 0 / Raw
<Sniffed: TCP:0 UDP:0 ICMP:4 Other:0>
>>> █
```

```
kali@kali:~$ ping 192.168.30.130
PING 192.168.30.130 (192.168.30.130) 56(84) bytes of data.
64 bytes from 192.168.30.130: icmp_seq=1 ttl=128 time=0.610 ms
64 bytes from 192.168.30.130: icmp_seq=2 ttl=128 time=1.08 ms
64 bytes from 192.168.30.130: icmp_seq=3 ttl=128 time=1.20 ms
64 bytes from 192.168.30.130: icmp_seq=4 ttl=128 time=1.04 ms
64 bytes from 192.168.30.130: icmp_seq=5 ttl=128 time=1.13 ms
^C
```

Sniff and the
output is
src, dst and
protocol

```
>>> sniff(prn = lambda x: \
... : x.strftime("%IP.src% > %IP.dst% %IP.proto%"))
192.168.30.130 > 8.8.4.4 udp
?? > ?? ??
?? > ?? ??
192.168.30.130 > 8.8.8.8 udp
192.168.30.130 > 8.8.4.4 udp
192.168.30.130 > 8.8.8.8 udp
192.168.30.130 > 8.8.4.4 udp
192.168.30.130 > 8.8.8.8 udp
192.168.30.130 > 8.8.4.4 udp
10.10.10.10 > 192.168.30.130 tcp
192.168.30.130 > 10.10.10.10 tcp
192.168.30.130 > 10.10.10.10 tcp
192.168.30.130 > 10.10.10.10 tcp
?? > ?? ??
?? > ?? ??
192.168.30.129 > 192.168.30.254 udp
192.168.30.254 > 192.168.30.129 udp
10.10.10.10 > 192.168.30.130 tcp
?? > ?? ??
?? > ?? ??
192.168.30.130 > 8.8.4.4 udp
192.168.30.130 > 8.8.8.8 udp
192.168.30.130 > 8.8.4.4 udp
^C<Sniffed: TCP:5 UDP:12 ICMP:0 Other:6>
>>>
```

Collect more information about the network

```
>>> sniff(iface="eth0", prn=lambda x: x.show())
###[ Ethernet ]###
  dst= 00:50:56:f4:90:b5
  src= 00:0c:29:77:46:96
  type= IPv4
###[ IP ]###
  version= 4
  ihl= 5
  tos= 0x0
  len= 96
  id= 1797
  flags=
  frag= 0
  ttl= 128
  proto= udp
```

Display source and destination IP address

```
>>> pkts = sniff(prn=lambda x:x.strftime("{IP:%IP.src% → %IP.dst% \n}{Raw:
... : %Raw.load%\n}"))
```

```
192.168.30.132 → 224.0.0.22
```

```
'"\x00\xfa\x01\x00\x00\x00\x01\x03\x00\x00\x00\xe0\x00\x00\xfc'
```

```
192.168.30.132 → 224.0.0.22
```

```
'"\x00\xfa\x01\x00\x00\x00\x01\x04\x00\x00\x00\xe0\x00\x00\xfc'
```

```
192.168.30.132 → 224.0.0.252
```

```
192.168.30.132 → 224.0.0.252
```

```
WARNING: DNS RR prematured end (ofs=3, len=3)
```

```
192.168.30.132 → 224.0.0.22
```

```
'"\x00\xea\x03\x00\x00\x00\x01\x04\x00\x00\x00\xef\xff\xff\xfa'
```

```
192.168.30.132 → 239.255.255.250
```

```
'<?xml version="1.0" encoding="utf-8"?><soap:Envelope xmlns:soap="http://w
```

Loop to display the source and destination

```
>>> srloop(IP(dst= 'www.google.com')/TCP())
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
RECV 1:
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
RECV 1:
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
RECV 1:
RECV 1: IP / TCP 142.250.102.103:http > 192.168.30.129:ftp_data SA / Paddi
ng
```

Scapy for Active Network Probing

```
>>> ans , unans = sr(IP(dst="142.250.102.103")/TCP(dport=[20, 21, 22, 25, 53, 80]))
Begin emission:
Finished sending 6 packets.
.*.....
.....
```

- Sr = send and receive
- Ans= when the message **get respond** put the output on **variable ans**
- Unans= when the send receive **get error** put it on **unans**
- Collect information form google ip address and used multi ports

Scapy for Active Network Probing

- Query to connect between IP address and ports

```
>>> ans.make_table(lambda s, r: (s.dst, s.dport, r.strftime("%IP.id%")))
      142.250.102.103
80 1960
>>> |
```

Send and
receive and
get the
summary

```
>>> sr(IP(dst="192.168.30.130", ttl=(10,20))/TCP(sport=RandShort()))
Begin emission:
Finished sending 11 packets.
.*****
Received 12 packets, got 11 answers, remaining 0 packets
(<Results: TCP:11 UDP:0 ICMP:0 Other:0>,
 <Unanswered: TCP:0 UDP:0 ICMP:0 Other:0>)
>>> res, unans = _
>>> res.summary()
IP / TCP 192.168.30.129:14558 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:14558 SA / Padding
IP / TCP 192.168.30.129:40258 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:40258 SA / Padding
IP / TCP 192.168.30.129:53430 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:53430 SA / Padding
IP / TCP 192.168.30.129:56348 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:56348 SA / Padding
IP / TCP 192.168.30.129:30521 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:30521 SA / Padding
IP / TCP 192.168.30.129:58514 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:58514 SA / Padding
IP / TCP 192.168.30.129:31642 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:31642 SA / Padding
IP / TCP 192.168.30.129:16638 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:16638 SA / Padding
IP / TCP 192.168.30.129:17590 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:17590 SA / Padding
IP / TCP 192.168.30.129:10652 > 192.168.30.130:http S => IP / TCP 192.168.30.130:http > 192.
168.30.129:10652 SA / Padding
```

Routing table

```
>>> conf.route
```

Network	Netmask	Gateway	Iface	Output IP	Metric
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	0
127.0.0.0	255.0.0.0	0.0.0.0	lo	127.0.0.1	1
192.168.30.0	255.255.255.0	0.0.0.0	eth0	192.168.30.129	0

```
>>> █
```

Add default gateway

```
>>> conf.route.add(net = "0.0.0.0/0" , gw="192.168.30.2")
>>> conf.route
```

Network	Netmask	Gateway	Iface	Output IP	Metric
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	0
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	1
127.0.0.0	255.0.0.0	0.0.0.0	lo	127.0.0.1	1
192.168.30.0	255.255.255.0	0.0.0.0	eth0	192.168.30.129	0

```
>>> 
```

Add host to routing table

```
>>> conf.route.add(host = "2.2.2.12" , gw="2.2.2.2")
>>> conf.route
```

Network	Netmask	Gateway	Iface	Output IP	Metric
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	0
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	1
127.0.0.0	255.0.0.0	0.0.0.0	lo	127.0.0.1	1
192.168.30.0	255.255.255.0	0.0.0.0	eth0	192.168.30.129	0
2.2.2.12	255.255.255.255	2.2.2.2	eth0	192.168.30.129	1

```
>>> 
```

Delete host from routing table

```
>>> conf.route.delt(host = "2.2.2.12" , gw="2.2.2.2")
>>> conf.route
```

Network	Netmask	Gateway	Iface	Output IP	Metric
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	0
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	1
127.0.0.0	255.0.0.0	0.0.0.0	lo	127.0.0.1	1
192.168.30.0	255.255.255.0	0.0.0.0	eth0	192.168.30.129	0

```
>>> 
```

Re- synchronace between table and the real table

```
>>> conf.route.resync()
```

```
>>> conf.route
```

Network	Netmask	Gateway	Iface	Output IP	Metric
0.0.0.0	0.0.0.0	192.168.30.2	eth0	192.168.30.129	0
127.0.0.0	255.0.0.0	0.0.0.0	lo	127.0.0.1	1
192.168.30.0	255.255.255.0	0.0.0.0	eth0	192.168.30.129	0

```
>>> 
```

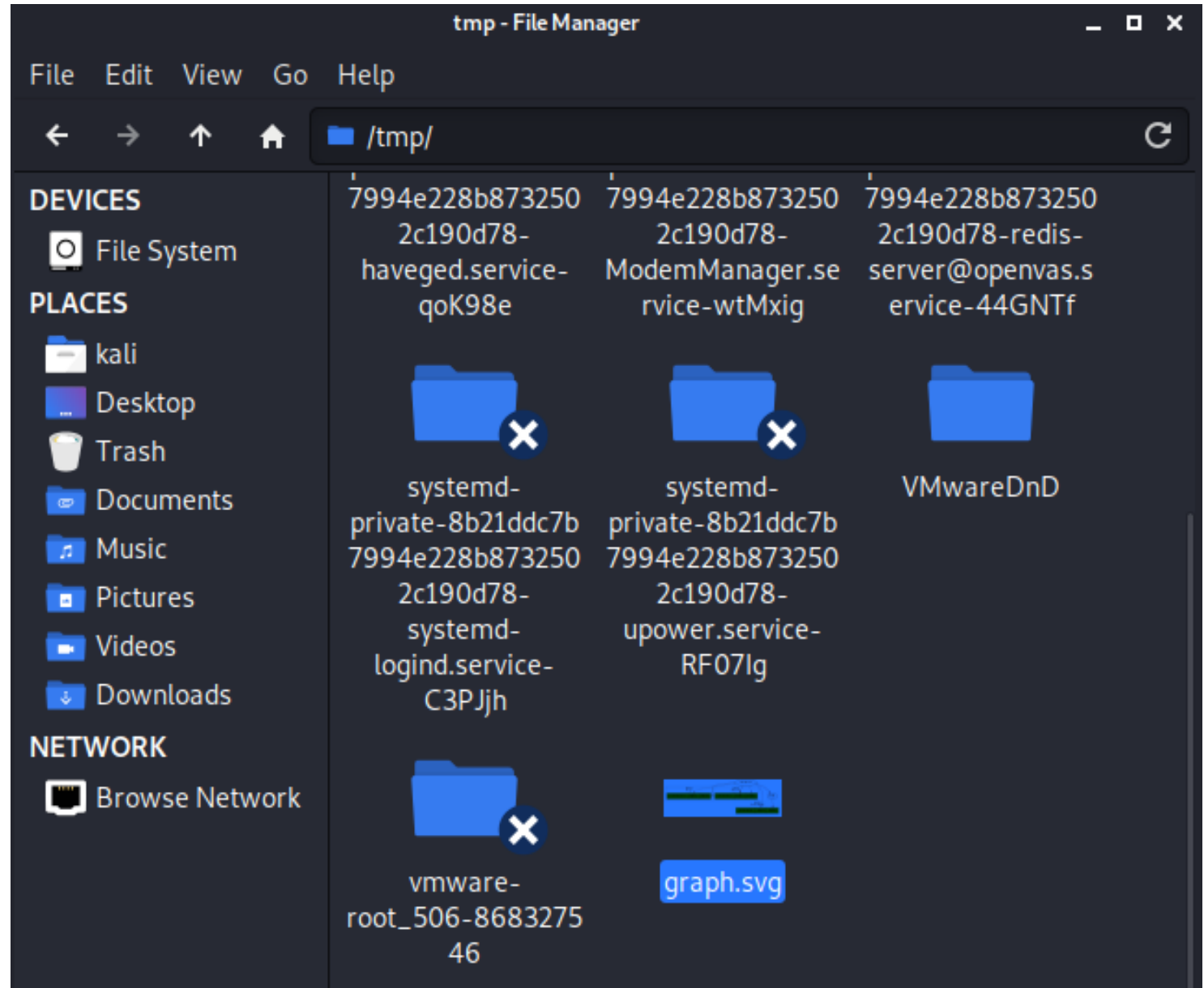
Make the output on variable

```
<Unfinished: TCP:0 UDP:0 ICMP:0 Other:0>
>>> res, unans = traceroute(["www.google.com", "www.facebook.com", "www.yahoo.com"],
... : maxttl=20, dport=[80, 443], retry = -2)
Begin emission:
Finished sending 120 packets.
*****
*****.....*****
Received 158 packets, got 120 answers, remaining 0 packets
  142.250.102.99:tcp443 142.250.102.99:tcp80 157.240.195.35:tcp443 157.240.195.35:tc
p80 87.248.100.215:tcp443 87.248.100.215:tcp80
1 192.168.30.2 11 192.168.30.2 11 192.168.30.2 11 192.168.30.2 1
1 192.168.30.2 11 192.168.30.2 11
2 142.250.102.99 SA 142.250.102.99 SA 157.240.195.35 SA 157.240.195.35 S
A 87.248.100.215 SA 87.248.100.215 SA
3 142.250.102.99 SA 142.250.102.99 SA 157.240.195.35 SA 157.240.195.35 S
A 87.248.100.215 SA 87.248.100.215 SA
4 142.250.102.99 SA 142.250.102.99 SA 157.240.195.35 SA 157.240.195.35 S
A 87.248.100.215 SA 87.248.100.215 SA
5 142.250.102.99 SA 142.250.102.99 SA 157.240.195.35 SA 157.240.195.35 S
A 87.248.100.215 SA 87.248.100.215 SA
6 142.250.102.99 SA 142.250.102.99 SA 157.240.195.35 SA 157.240.195.35 S
A 87.248.100.215 SA 87.248.100.215 SA
7 142.250.102.99 SA 142.250.102.99 SA 157.240.195.35 SA 157.240.195.35 S
A 87.248.100.215 SA 87.248.100.215 SA
```

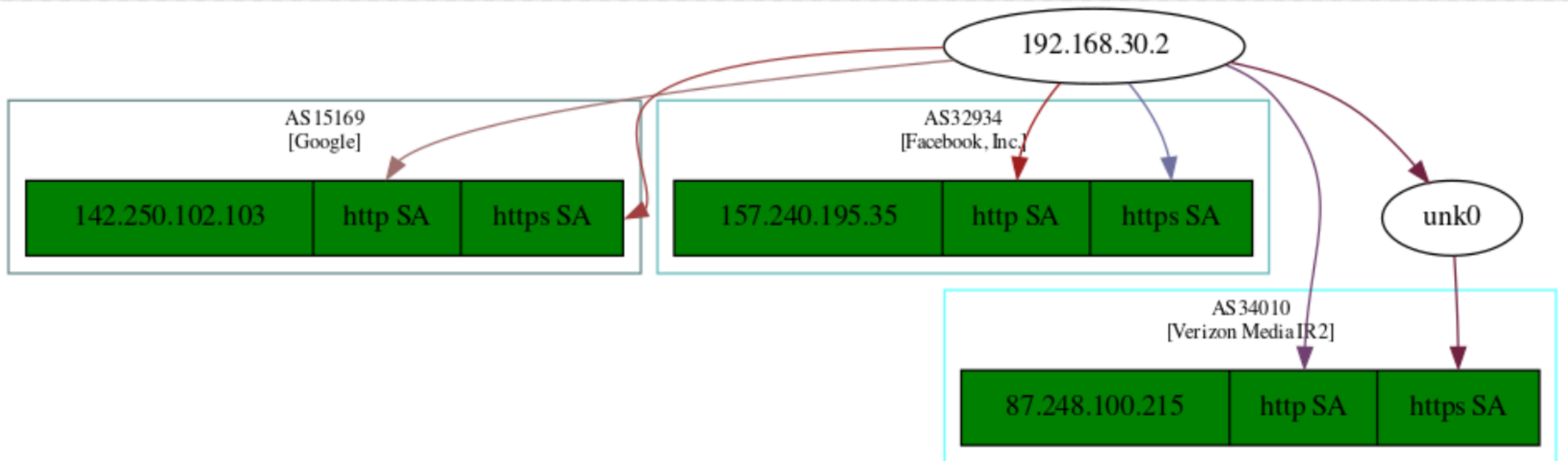
Save the graph on files

```
>>> res.graph(target="> /tmp/graph.svg")  
>>> |
```

Save on folder



Open the graph



Send receive with scapy

```
>>> help(sr)
...:
...:

>>> ip=IP(src="1.1.1.1")
>>> ip.dst="10.10.10.133"
>>> ip
<IP  src=1.1.1.1 dst=10.10.10.133 >
>>> tcp=TCP(sport=2600, dport=80)
>>> sr(ip/tcp)
Begin emission:
Finished sending 1 packets.
... *
Received 4 packets, got 1 answers, remaining 0 packets
(<Results: TCP:1 UDP:0 ICMP:0 Other:0>,
 <Unanswered: TCP:0 UDP:0 ICMP:0 Other:0>)
```

Monitor host 10.10.10.133

```
kali@kali:~$ sudo tcpdump host 10.10.10.133
[sudo] password for kali:
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
04:32:07.091968 ARP, Request who-has 10.10.10.133 tell 10.10.10.131, length 28
04:32:07.092476 ARP, Reply 10.10.10.133 is-at 00:0c:29:98:c6:e4 (oui Unknown), length 46
04:32:07.123173 IP 1.1.1.1.zebrasrv > 10.10.10.133.http: Flags [S], seq 0, win 8192, length 0
04:32:07.123547 IP 10.10.10.133.http > 1.1.1.1.zebrasrv: Flags [S.], seq 3270774037, ack 1, win 5840, options [mss 1460], length 0
04:32:07.123547 ARP, Request who-has 10.10.10.133 tell 10.10.10.2, length 46
04:32:07.123754 ARP, Reply 10.10.10.133 is-at 00:0c:29:98:c6:e4 (oui Unknown), length 46
04:32:07.124230 IP 1.1.1.1.zebrasrv > 10.10.10.133.http: Flags [R], seq 1, win 32767, length 0
```

```
□
```

Spoofing

- **Decoy:**
 - **nmap -Pn -D <spoofed IP> <target>**
 - This will perform a spoofed ping scan.
- **Source Address Spoofing:**
 - **nmap -e <network interface> -S <IP source> <target>**
 - Example --> **nmap -e eth0 -S 10.0.0.140 10.0.0.165**
- **MAC Address Spoofing:**
 - **nmap --spoof-mac <MAC|Vendor> <target>**
 - Example --> **nmap --spoof-mac Cis 10.0.0.140**
- **Note: Decoys will send spoofed IP address along with your IP address.**

NMAP

spoof-mac

```
(kali@kali)-[~]
└─$ sudo nmap --spoof-mac 0 -e eth0 192.168.1.130
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-
Spoofing MAC address E3:29:85:FA:A9:D1 (No register
Nmap scan report for 192.168.1.130
Host is up (0.00086s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 00:0C:29:4B:36:0A (VMware)
```

Mac
address
before
spoofed

No.	Time	Source	Destination	Protocol	Length
3096	300.357018709	192.168.1.128	192.168.1.130	TCP	74
3097	300.357081328	192.168.1.128	192.168.1.130	TCP	74
3098	300.357109390	192.168.1.128	192.168.1.130	TCP	74
3099	300.357172574	192.168.1.128	192.168.1.130	TCP	74
3100	300.357201615	192.168.1.128	192.168.1.130	TCP	74
3101	300.357264181	192.168.1.128	192.168.1.130	TCP	74
3102	300.357293351	192.168.1.128	192.168.1.130	TCP	74
3103	300.357356738	192.168.1.128	192.168.1.130	TCP	74
3104	300.357385832	192.168.1.128	192.168.1.130	TCP	74
3105	300.357491881	192.168.1.130	192.168.1.128	TCP	60
3106	300.357491986	192.168.1.130	192.168.1.128	TCP	60
3107	300.357492061	192.168.1.130	192.168.1.128	TCP	60
3108	300.357492135	192.168.1.130	192.168.1.128	TCP	60
3109	300.357492210	192.168.1.130	192.168.1.128	TCP	60
3110	300.357492285	192.168.1.130	192.168.1.128	TCP	60
3111	300.357492359	192.168.1.130	192.168.1.128	TCP	60

Frame 3098: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface
Ethernet II, Src: VMware_5a:97:a8 (00:0c:29:5a:97:a8), Dst: VMware_4b:36:0a
Destination: VMware_4b:36:0a (00:0c:29:4b:36:0a)
Source: VMware_5a:97:a8 (00:0c:29:5a:97:a8) ✓
Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.1.128, Dst: 192.168.1.130
Transmission Control Protocol, Src Port: 40906, Dst Port: 7676, Seq: 0, Len: 74

Mac address after spoofed

No.	Time	Source	Destination	Protocol	Length	Info
5848	454.814043710	192.168.1.128	192.168.1.130	TCP	58	64480 → 4998 [SYN] Seq=0
5849	454.814142200	192.168.1.128	192.168.1.130	TCP	58	64480 → 7800 [SYN] Seq=0
5850	454.814165250	192.168.1.128	192.168.1.130	TCP	58	64480 → 1533 [SYN] Seq=0
5851	454.814262427	192.168.1.128	192.168.1.130	TCP	58	64480 → 5440 [SYN] Seq=0


```

▶ Frame 5849: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface eth
▼ Ethernet II, Src: e3:29:85:fa:a9:d1 (e3:29:85:fa:a9:d1), Dst: VMware_4b:36:0a (00:0c:2
  ▶ Destination: VMware_4b:36:0a (00:0c:29:4b:36:0a)
  ▶ Source: e3:29:85:fa:a9:d1 (e3:29:85:fa:a9:d1)
    Type: IPv4 (0x0800)
▶ Internet Protocol Version 4, Src: 192.168.1.128, Dst: 192.168.1.130
▶ Transmission Control Protocol, Src Port: 64480, Dst Port: 7800, Seq: 0, Len: 0
  
```

0000	00 0c 29
0010	00 2c 29
0020	01 82 fb
0030	04 00 2c

Spoofed source
IP address

```
(kali@kali)-[~]
$ sudo nmap -D RND:5 192.168.1.130
Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for 192.168.1.130
Host is up (0.0019s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 00:0C:29:4B:36:0A (VMware)
```

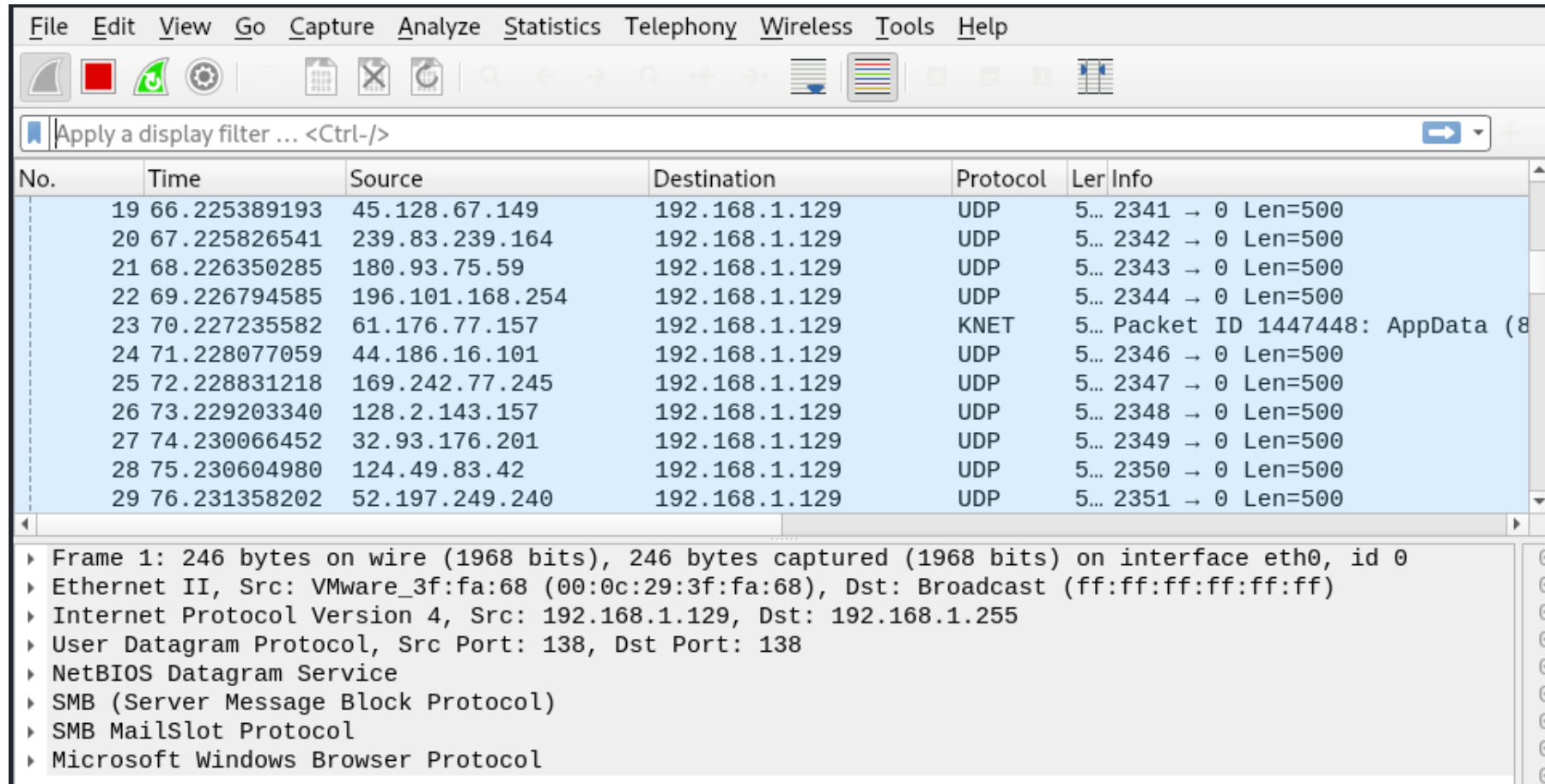
Used 5 IP address as source scan

No.	Time	Source	Destination	Protocol	Length	Info
7117	26.244114694	192.168.1.128	192.168.1.130	TCP	54	62131 → 5900 [RST] Seq=1 W
7118	26.244256872	192.168.1.128	192.168.1.130	TCP	58	62131 → 443 [SYN] Seq=0 Wi
7119	26.244401208	146.238.36.34	192.168.1.130	TCP	58	62131 → 443 [SYN] Seq=0 Wi
7120	26.244493023	2.93.105.85	192.168.1.130	TCP	58	62131 → 443 [SYN] Seq=0 Wi
7121	26.244637826	117.28.88.100	192.168.1.130	TCP	58	62131 → 443 [SYN] Seq=0 Wi
7122	26.244713680	192.168.1.130	192.168.1.128	TCP	60	443 → 62131 [RST, ACK] Seq
7123	26.244760028	23.169.110.146	192.168.1.130	TCP	58	62131 → 443 [SYN] Seq=0 Wi
7124	26.244884080	75.116.222.196	192.168.1.130	TCP	58	62131 → 443 [SYN] Seq=0 Wi
7125	26.245062544	192.168.1.128	192.168.1.130	TCP	58	62131 → 113 [SYN] Seq=0 Wi
7126	26.245187149	146.238.36.34	192.168.1.130	TCP	58	62131 → 113 [SYN] Seq=0 Wi
7127	26.245391972	192.168.1.130	192.168.1.128	TCP	60	113 → 62131 [RST, ACK] Seq
7128	26.245399713	2.93.105.85	192.168.1.130	TCP	58	62131 → 113 [SYN] Seq=0 Wi
7129	26.245552984	117.28.88.100	192.168.1.130	TCP	58	62131 → 113 [SYN] Seq=0 Wi
7130	26.245651079	23.169.110.146	192.168.1.130	TCP	58	62131 → 113 [SYN] Seq=0 Wi
7131	26.245774046	75.116.222.196	192.168.1.130	TCP	58	62131 → 113 [SYN] Seq=0 Wi
7132	26.245868257	192.168.1.128	192.168.1.130	TCP	58	62131 → 1025 [SYN] Seq=0 W

Hping3 with packet crafting random source ip address

```
(kali㉿kali)-[~]
└─$ sudo hping3 192.168.1.129 --udp --rand-source --data 500
[sudo] password for kali:
HPING 192.168.1.129 (eth0 192.168.1.129): udp mode set, 28 headers + 500 data bytes
^C
— 192.168.1.129 hping statistic —
44 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Random source IP address



The image shows a Wireshark network traffic capture. The main display area shows a list of 11 network packets. The 'Source' column for these packets contains various random IP addresses. The 'Destination' column for all packets is 192.168.1.129. The 'Protocol' column shows a mix of UDP and KNET. The 'Info' column provides details for each packet, including port numbers and lengths. The bottom pane shows the details of the first packet, which is a NetBIOS Datagram Service packet over SMB.

No.	Time	Source	Destination	Protocol	Len	Info
19	66.225389193	45.128.67.149	192.168.1.129	UDP	5...	2341 → 0 Len=500
20	67.225826541	239.83.239.164	192.168.1.129	UDP	5...	2342 → 0 Len=500
21	68.226350285	180.93.75.59	192.168.1.129	UDP	5...	2343 → 0 Len=500
22	69.226794585	196.101.168.254	192.168.1.129	UDP	5...	2344 → 0 Len=500
23	70.227235582	61.176.77.157	192.168.1.129	KNET	5...	Packet ID 1447448: AppData (8
24	71.228077059	44.186.16.101	192.168.1.129	UDP	5...	2346 → 0 Len=500
25	72.228831218	169.242.77.245	192.168.1.129	UDP	5...	2347 → 0 Len=500
26	73.229203340	128.2.143.157	192.168.1.129	UDP	5...	2348 → 0 Len=500
27	74.230066452	32.93.176.201	192.168.1.129	UDP	5...	2349 → 0 Len=500
28	75.230604980	124.49.83.42	192.168.1.129	UDP	5...	2350 → 0 Len=500
29	76.231358202	52.197.249.240	192.168.1.129	UDP	5...	2351 → 0 Len=500

Frame 1: 246 bytes on wire (1968 bits), 246 bytes captured (1968 bits) on interface eth0, id 0

- Ethernet II, Src: VMware_3f:fa:68 (00:0c:29:3f:fa:68), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
- Internet Protocol Version 4, Src: 192.168.1.129, Dst: 192.168.1.255
- User Datagram Protocol, Src Port: 138, Dst Port: 138
- NetBIOS Datagram Service
- SMB (Server Message Block Protocol)
- SMB MailSlot Protocol
- Microsoft Windows Browser Protocol

Spoofing IP address

- **-a** 192.168.1.190 determine the **source IP address**, this for kali1 system, this is the **spoofed source IP address**
- 192.168.1.129 this is for the **victim** this win xp system

```
(kali@kali)-[~/Desktop/data]
└─$ sudo hping3 -S -a 192.168.1.190 --flood -p 80 192.168.1.129
[sudo] password for kali:
HPING 192.168.1.129 (eth0 192.168.1.129): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
— 192.168.1.129 hping statistic —
1117759 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

From kali1

- This from the **spoofed IP address** from kali1 system

Access documents, folders and network places

Capturing from eth0 [Wireshark 1.10.2 (SVN Rev 51934 from /trunk-1.10)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: Expression... Clear Apply Save

No.	Time	Source	Destination	Protocol	Length	Info
14707	83.240664000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14708	83.240669000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14709	83.240670000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14710	83.240671000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14711	83.240672000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14712	83.240673000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14713	83.240748000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14714	83.240750000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14715	83.240750000	192.168.1.129	192.168.1.190	TCP	60	[TCP ACKed unseen segment] http > x11 [RST, ACK] Seq=1 Ack=1353875151 Win=0 Len=0
14716	83.240820000	192.168.1.129	192.168.1.190	TCP	60	http > x11 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Spoof source IP address

```
(kali@kali)-[~]
$ sudo hping3 192.168.1.129 --udp --spoof 192.168.1.190 --data 500
HPING 192.168.1.129 (eth0 192.168.1.129): udp mode set, 28 headers + 500 data bytes
^C* (wireshark:8644) 08:06:56.949806 [GUI WARNING] -- QStandardPaths: XDG_RUNTIME_DI
—o192.168.1.129 hping statistic—time-root'
33 packets transmitted, 0 packets received, 100% packet loss
capture Start ...
round-trip min/avg/max = 0.0/0.0/0.0 ms
capture MESSAGE] -- Capture started
** (wireshark:8644) 08:07:04.202734 [Capture MESSAGE] -- File: "/tmp/wireshark_eth0
(kali@kali)-[~]
$
```

Spoofed IP address kali1

No.	Time	Source	Destination	Protocol	Length	Info
11	12.923205000	192.168.1.190	192.168.1.129	UDP	542	Source port: conferencetalk Destination port: 0
12	13.923938000	192.168.1.190	192.168.1.129	UDP	542	Source port: sesi-lm Destination port: 0
13	14.924451000	192.168.1.190	192.168.1.129	UDP	542	Source port: houdini-lm Destination port: 0
14	15.925425000	192.168.1.190	192.168.1.129	UDP	542	Source port: xmsg Destination port: 0
15	16.925765000	192.168.1.190	192.168.1.129	UDP	542	Source port: fj-hdnet Destination port: 0
16	17.926587000	192.168.1.190	192.168.1.129	H.225.0	542	Source port: h323gatedisc Destination port: 0[UNKNOWN PER: too many extensions][Malformed Packet]
17	18.927369000	192.168.1.190	192.168.1.129	H.225.0	542	Source port: h323gatestat Destination port: 0[UNKNOWN PER: too many extensions][Malformed Packet]
18	19.928075000	192.168.1.190	192.168.1.129	UDP	542	Source port: h323hostcall Destination port: 0
19	20.928997000	192.168.1.190	192.168.1.129	UDP	542	Source port: caicci Destination port: 0

Firewall / IDS Evasion and Spoofing

SWITCH	EXAMPLE	DESCRIPTION
-f	<code>nmap 192.168.1.1 -f</code>	Requested scan (including ping scans) use tiny fragmented IP packets. Harder for packet filters
-mtu	<code>nmap 192.168.1.1 -mtu 32</code>	Set your own offset size
-D	<code>nmap -D 192.168.1.101,192.168.1.102,192.168.1.103,192.168.1.23 192.168.1.1</code>	Send scans from spoofed IPs
-D	<code>nmap -D decoy-ip1,decoy-ip2,your-own-ip,decoy-ip3,decoy-ip4 remote-host-ip</code>	Above example explained
-S	<code>nmap -S www.microsoft.com www.facebook.com</code>	Scan Facebook from Microsoft (-e eth0 -Pn may be required)
-g	<code>nmap -g 53 192.168.1.1</code>	Use given source port number
-proxies	<code>nmap -proxies http://192.168.1.1:8080, http://192.168.1.2:8080 192.168.1.1</code>	Relay connections through HTTP/SOCKS4 proxies
-data-length	<code>nmap -data-length 200 192.168.1.1</code>	Appends random data to sent packets

SCTP INIT Ping

This **discovery method attempts to locate hosts using** the Stream Control Transmission Protocol (SCTP). SCTP is typically used on systems for **IP based telephony**.

```
# nmap -PY 192.168.1.254
```

The **default port for -PY is 80**. Others can be specified by using the following syntax:

nmap -PY22,25,80,443

ICMP Echo Ping

```
# nmap -PE 192.168.1.254
```

works best on local networks where ICMP packets can be transmitted with few restrictions. The **-PE option is automatically implied if no other ping options are specified**.

```
(kali㉿kali)-[~]
$ sudo nmap -PE 192.168.1.194
Starting Nmap 7.93 ( https://nmap.org ) at 2023-01-31 07:31 EST
Nmap scan report for 192.168.1.194
Host is up (0.00090s latency).
All 1000 scanned ports on 192.168.1.194 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:0C:29:3F:FA:68 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 27.23 seconds

(kali㉿kali)-[~]
$
```

ICMP Timestamp Ping

```
# nmap -PP 192.168.1.254
```

While most **firewalled systems** are configured to **block ICMP echo requests**, some improperly configured systems may still **reply to ICMP timestamp requests**. This **makes -PP useful for attempting to solicit responses from firewalled targets**.

ICMP Address Mask Ping

```
# nmap -PM 192.168.1.254
```

similar to the -PP option

This type of ping can occasionally sneak past a firewall that is configured to **block standard echo requests**.

IP Protocol Ping

```
# nmap -PO 10.10.1.48
```

An IP protocol ping sends packets with the specified protocol to the target. If no protocols are specified the **default protocols 1 (ICMP), 2 (IGMP), and 4 (IP-in-IP)** are used. To ping using a custom set of protocols, use the following syntax:

nmap -PO1,2,4

Bypass the firewall on win XP

```
(kali®kali)-[~]  
$ sudo nmap -sF -p1-100 -T4 192.168.1.129  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-26 08:32 EST  
Nmap scan report for 192.168.1.129  
Host is up (0.00026s latency).  
All 100 scanned ports on 192.168.1.129 are in ignored states.  
Not shown: 100 open|filtered tcp ports (no-response)  
MAC Address: 00:0C:29:3F:FA:68 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 14.22 seconds  
  
(kali®kali)-[~]  
$
```

Bypass the firewall on win XP

```
(kali㉿kali)-[~]  
$ sudo nmap -sS -v -v -Pn 192.168.1.129  
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-26 08:34 EST  
Initiating ARP Ping Scan at 08:34  
Scanning 192.168.1.129 [1 port]  
Completed ARP Ping Scan at 08:34, 0.04s elapsed (1 total hosts)  
Initiating Parallel DNS resolution of 1 host. at 08:34  
Completed Parallel DNS resolution of 1 host. at 08:34, 11.02s elapsed  
Initiating SYN Stealth Scan at 08:34  
Scanning 192.168.1.129 [1000 ports]  
Completed SYN Stealth Scan at 08:35, 21.18s elapsed (1000 total ports)  
Nmap scan report for 192.168.1.129  
Host is up, received arp-response (0.00026s latency).  
Scanned at 2023-12-26 08:34:53 EST for 21s  
All 1000 scanned ports on 192.168.1.129 are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
MAC Address: 00:0C:29:3F:FA:68 (VMware)  
  
Read data files from: /usr/bin/../../share/nmap  
Nmap done: 1 IP address (1 host up) scanned in 32.36 seconds  
Raw packets sent: 2001 (88.028KB) | Rcvd: 1 (28B)
```

Bypass the firewall on win XP

```
(kali㉿kali)-[~]  
$ sudo nmap -n -sn -PE -T4 192.168.1.129  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-26 08:37 EST  
Nmap scan report for 192.168.1.129  
Host is up (0.00024s latency).  
MAC Address: 00:0C:29:3F:FA:68 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
```

```
(kali㉿kali)-[~]  
$
```

Bypass the firewall on win XP

```
(kali㉿kali)-[~]  
$ sudo nmap --randomize-hosts 192.168.1.129  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-26 08:44 EST  
Nmap scan report for 192.168.1.129  
Host is up (0.00030s latency).  
All 1000 scanned ports on 192.168.1.129 are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
MAC Address: 00:0C:29:3F:FA:68 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 32.37 seconds
```

Bypass the firewall on win XP

```
(kali@kali)-[~]  
$ sudo nmap -sS -T5 192.168.1.129 --script firewall-bypass  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-26 08:47 EST  
Nmap scan report for 192.168.1.129  
Host is up (0.00023s latency).  
All 1000 scanned ports on 192.168.1.129 are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
MAC Address: 00:0C:29:3F:FA:68 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 23.41 seconds
```

Force Reverse DNS Resolution

```
# nmap -R 64.13.134.52
```

```
Starting Nmap 5.00 ( http://nmap.org ) at 2009-08-13 17:22 Central  
Daylight Time
```

```
Interesting ports on scanme.nmap.org (64.13.134.52):
```

```
Not shown: 993 filtered ports
```

PORT	STATE	SERVICE
------	-------	---------

25/tcp	closed	smtp
--------	--------	------

53/tcp	open	domain
--------	------	--------

70/tcp	closed	gopher
--------	--------	--------

80/tcp	open	http
--------	------	------

110/tcp	closed	pop3
---------	--------	------

113/tcp	closed	auth
---------	--------	------

31337/tcp	closed	Elite
-----------	--------	-------

```
Nmap done: 1 IP address (1 host up) scanned in 9.38 seconds
```

Firewall is active

```
(kali@kali)-[~]  
$ sudo nmap -R 192.168.1.129  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2023-12-26 08:51 EST  
Nmap scan report for 192.168.1.129  
Host is up (0.00025s latency).  
All 1000 scanned ports on 192.168.1.129 are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
MAC Address: 00:0C:29:3F:FA:68 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 32.34 seconds
```

useful when performing reconnaissance on a **block of IP addresses** as Nmap will try to resolve the **reverse DNS** information of every IP address.

But dramatically reduce the performance of a scan.

Disable Reverse DNS Resolution

```
# nmap -n 64.13.134.52

Starting Nmap 5.00 ( http://nmap.org ) at 2009-08-13 17:23 Central
Daylight Time

Interesting ports on 64.13.134.52:
Not shown: 993 filtered ports
PORT      STATE SERVICE
25/tcp    closed smtp
53/tcp    open  domain
70/tcp    closed gopher
80/tcp    open  http
110/tcp   closed pop3
113/tcp   closed auth
31337/tcp closed Elite

Nmap done: 1 IP address (1 host up) scanned in 8.48 seconds
```

greatly reduces scanning times

Traceroute

```
(kali㉿kali)-[~]  
$ sudo nmap -traceroute facebook.com  
[sudo] password for kali:  
Starting Nmap 7.93 ( https://nmap.org ) at 2023-01-31 09:20 EST  
Nmap scan report for facebook.com (157.240.195.35)  
Host is up (0.010s latency).  
Other addresses for facebook.com (not scanned): 2a03:2880:f142:182:face:b00c:0:25de  
rDNS record for 157.240.195.35: edge-star-mini-shv-01-mrs2.facebook.com  
Not shown: 997 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
53/tcp    open  domain  
80/tcp    open  http  
443/tcp   open  https  
  
TRACEROUTE (using port 80/tcp)  
HOP RTT      ADDRESS  
1    0.24 ms 192.168.8.2  
2    0.24 ms edge-star-mini-shv-01-mrs2.facebook.com (157.240.195.35)  
  
Nmap done: 1 IP address (1 host up) scanned in 63.52 seconds
```

Homework

- Which Web Application Firewall (WAF) and how can I use it
- How I can use the firewall Evasion techniques? Give me some practical examples?
- Compare between nmap and scapy? With practical
- **Nuclei**
- NetExec
- Scapy + AI Use AI-Assisted Network Packet Analysis for Scapy → Capture packets → Extract fields → AI analysis
- Practically Investigating a Security Incident with Microsoft Security Copilot.
- AI + MITRE ATT&CK